Contents

Altium Schematic Instructions — Flight Controller Board

Design Summary

This board implements the CubeSat's flight-controller (system orchestrator) on a single PCB, using a Raspberry Pi Pico (RP2040) module as the MCU. It is a power consumer on the CSKB stack — no on-board regulators — and acts as command authority, mode-state owner, and telemetry aggregator for the rest of the satellite.

```
CubeSat Kit Bus (CSKB) Stack Connectors — H1 (signals) + H2 (power)
     │
     ├── +5V_SYS (H2.25/26) ──→ Schottky ──→ Pico VSYS ──→ Pico internal LDO ──→ on-
module 3V3
     ├── VCC = +3V3 (H2.27/28) ─→ optional / not used by Pico in v0.1
     ├── DGND (H2.29/30/32), AGND (H2.31)
     ├── VBATT (H2.45/46) ──→ NOT USED (EPS owns battery telemetry)
     ├── SDA_SYS / SCL_SYS (H1.41/43) ──→ I2C to EPS LTC4162
     ├── IO.0..IO.3 (H1.24/23/22/21) ──→ SPI0 to Comms RP2040 (FC = master)
     ├── IO.4 / IO.5 (H1.20/19) ──→ UART0 debug TX/RX
     ├── IO.8 (H1.16) ──→ COMMS_IRQ (comms → FC, data-ready)
     ├── USER0..USER3 (H1.47/48/49/50) ──→ COMMS_EN, COMMS_FAULT_N, EPS_ALERT_N, PAYLOAD_EN
     ├── USER4 / USER5 (H1.51/52) ──→ CAN_H / CAN_L (Iter 2 stub via MAX3051)
     └── -RESET (H1.29) ──→ tied to STWD100 WDO_N and Pico RUN

RP2040 (Raspberry Pi Pico module)
     │
     ├── I2C0 (GP4/GP5) ──→ EPS LTC4162 housekeeping
     ├── SPI0 (GP16-19) ──→ Comms RP2040 (FC is master)
```

```
     ├── SPI1 (GP10-13) ──→ MR25H40 4Mbit SPI MRAM (persistent state)
     ├── UART0 (GP0/GP1) ──→ Debug header
     ├── GP6  ←── COMMS_IRQ (input, internal pull-up)
     ├── GP7  ──→ COMMS_EN
     ├── GP8  ←── COMMS_FAULT_N
     ├── GP9  ←── EPS_ALERT_N
     ├── GP2  ──→ PAYLOAD_EN
     ├── GP22 ──→ WDT_FEED ──→ STWD100 WDI
     ├── GP20/GP21 ──→ LED_HEARTBEAT, LED_FAULT
     └── GP14/GP15 ──→ CAN_CS_N, CAN_INT (reserved for Iter 2 MCP2515 stub)

STWD100NYWY3F Hardware Watchdog
     │
     ├── WDI ←── GP22 (must be toggled within window)
     ├── WDO_N ──→ Pico RUN (resets MCU on timeout)
     └── EN_N ── JP1 "RBF / Disable Watchdog" jumper
                 (jumper installed = WDT disabled, bench mode;
                  jumper REMOVED = WDT enabled, FLIGHT mode)

MR25H40 4Mbit SPI MRAM
     │
     └── SPI1 ←── persistent boot counter, mode state, fault log
```

Key design decisions:

1. **Power comes from the EPS via the CSKB stack connectors** — no on-board voltage regulators. +5V_SYS feeds Pico VSYS through a Schottky diode. The Pico's internal buck-boost regulator produces 3V3 on-module. This matches the comms board pattern.
2. **Raspberry Pi Pico module** for v0.1 — bare RP2040 deferred to v2. Same rationale as the comms board: keeps support circuitry off the main PCB until the architecture is validated.
3. **MR25H40 SPI MRAM** for persistent storage, mirroring the AMSAT RT-IHU (Radiation Tolerant Internal Housekeeping Unit) pattern. MRAM has unlimited write endurance, no wear levelling required, and is the canonical CubeSat NV memory choice. 4 Mbit is more than enough for boot counters, mode state, and fault logs.
4. **External hardware watchdog (STWD100NYWY3F)** with a removable "Disable Watchdog" jumper (`JP1`). Jumper installed = bench/debug mode (WDT disabled), jumper removed = flight mode (WDT enabled). This matches the RT-IHU's RBF-style WDT enable jumper. WDO_N drives the Pico RUN pin so a missed feed forces a hard reset.
5. **No on-board RTC** — matches RT-IHU. Mission time is maintained as an MCU uptime counter, persisted to MRAM at boot, and ground-synced over the uplink command channel.
6. **CSKB pin map adopted verbatim from the Pumpkin CubeSat Kit Bus** — see `DS_CSK_MB_710-00484-E.pdf` (Pumpkin Motherboard Rev. E, doc Rev. A, March 2012), pages 13–16 for the canonical H1/H2 pin tables. This makes the centisat stack mechanically and electrically interop-

erable with Pumpkin and iSpace CSKB-compliant boards if we ever want to swap any subsystem for a commercial unit. The canonical pin map lives at `system/interfaces/cskb_pinmap.md`.

7. **CAN is stubbed only** in v0.1: MAX3051 transceiver footprint with slew resistor selecting 500 kbps (matches RT-IHU). The MCP2515 SPI-CAN controller and crystal are NOT populated in v0.1 — footprints reserved for Iteration 2.
8. **EPS owns battery telemetry.** The FC does NOT have its own VBAT divider into an ADC pin — it consumes battery voltage and current data from the EPS LTC4162 over I2C. VBATT pin on the stack is wired but unused on this board.
9. **FC is SPI master** to the comms board. The comms board's RP2040 is the slave and asserts a `COMMS_IRQ` output when it has a packet ready for the FC. This mirrors AMSAT RT-IHU (`MRAMDev` and `DCTDev` are both slaves to the IHU MCU; the AX5043 IRQ pin signals the IHU).
10. **Sheet count is six.** Same flat (non-hierarchical) design as the EPS and comms boards — net labels and power ports connect across sheets without sheet symbols or ports.

Altium Project Setup

1. Create a new PCB project: `Flight_Controller_Board.PrjPcb`
2. Add six schematic sheets (flat design):
   - `Overview.SchDoc` — title sheet, block diagram, design notes (non-electrical)
   - `MCU_Core.SchDoc` — Pico module, decoupling, status LEDs
   - `Memory_WDT.SchDoc` — MR25H40 MRAM, STWD100 watchdog, RBF jumper
   - `Interfaces.SchDoc` — I2C to EPS, SPI to comms, UART debug, control/fault GPIOs, CAN stub
   - `Power.SchDoc` — input protection, +5V→VSYS Schottky, bypass
   - `Connectors_Debug.SchDoc` — CSKB H1 + H2 stack connectors (Pumpkin pin map), debug header, spare GPIO header
3. Each sheet uses A3 landscape format
4. Set the project's designator scope to "Flat" so R1, C1, U1, etc. are unique across all sheets

Flat Design — How Inter-Sheet Connections Work

This project uses a **flat** (non-hierarchical) design, same as the EPS and comms boards. There is no top-level sheet with sheet symbols. Instead, nets connect between sheets using **net labels** and **power ports**:

- **Power ports** (`+3V3`, `+5V_SYS`, `GND`, `VBAT`) are global automatically — place the same power port symbol on any sheet and they connect.
- **Signal net labels** (`I2C_SDA`, `SPI_COMMS_SCK`, `WDT_FEED`, etc.) are also global in a flat project — place the same net label on two different sheets and Altium connects them automatically.
- **No ports or sheet entries needed** — those are only for hierarchical designs.

Net Name Convention

Use these global net names consistently across all sheets. Use **power port symbols** (not net labels) for supply rails so they are global automatically.

Net Name	Type	Pumpkin CSKB pin	Description
`+5V_SYS`	Power port	H2.25, H2.26	5.0V system rail (from EPS)
`+3V3`	Power port	(Pico-internal LDO output)	3.3V on-module rail (NOT routed off-module)
`VCC_BUS`	Power port	H2.27, H2.28	Pumpkin VCC (=3V3 bus rail), reserved, not used in v0.1
`VBAT`	Power port	H2.45, H2.46	Battery bus, monitor only — not used on FC
`GND`	Power port	H2.29, H2.30, H2.32	Digital ground (global)
`AGND`	Power port	H2.31	Analog ground, single-point tied to GND (single pin on CSKB)
`VSYS`	Net label	(internal)	Pico VSYS pin, fed from +5V_SYS via Schottky
`I2C_SDA`	Net label	H1.41 (`SDA_SYS`)	I2C data — to EPS LTC4162
`I2C_SCL`	Net label	H1.43 (`SCL_SYS`)	I2C clock — to EPS LTC4162
`SPI_COMMS_SCK`	Net label	H1.21 (`IO.3`)	SPI clock to comms board (FC = master)
`SPI_COMMS_MOSI`	Net label	H1.23 (`IO.1`)	SPI data, FC → comms
`SPI_COMMS_MISO`	Net label	H1.22 (`IO.2`)	SPI data, comms → FC
`SPI_COMMS_CS_N`	Net label	H1.24 (`IO.0`)	SPI chip select to comms (active-low)
`COMMS_IRQ`	Net label	H1.16 (`IO.8`)	Comms → FC data-ready interrupt (active-low)
`UART_DBG_TX`	Net label	H1.20 (`IO.4`)	Debug UART TX from FC
`UART_DBG_RX`	Net label	H1.19 (`IO.5`)	Debug UART RX into FC

Net Name	Type	Pumpkin CSKB pin	Description
COMMS_EN	Net label	H1.47 (USER0)	FC → comms enable (active-high)
COMMS_FAULT_N	Net label	H1.48 (USER1)	Comms → FC fault (active-low)
EPS_ALERT_N	Net label	H1.49 (USER2)	EPS → FC SMBus alert (active-low)
PAYLOAD_EN	Net label	H1.50 (USER3)	FC → payload enable
CAN_H	Net label	H1.51 (USER4)	CAN bus high (Iter 2 stub)
CAN_L	Net label	H1.52 (USER5)	CAN bus low (Iter 2 stub)
RESET_N	Net label	H1.29 (-RESET)	Stack reset, tied to Pico RUN
MRAM_SCK	Net label	(board-local)	SPI1 clock to MR25H40
MRAM_MOSI	Net label	(board-local)	SPI1 data, FC → MRAM
MRAM_MISO	Net label	(board-local)	SPI1 data, MRAM → FC
MRAM_CS_N	Net label	(board-local)	SPI1 chip select to MRAM
WDT_FEED	Net label	(board-local)	Pico → STWD100 WDI
WDT_RST_N	Net label	(board-local)	STWD100 WDO_N → Pico RUN (open-drain, active-low)
LED_HB	Net label	(board-local)	GPIO → heartbeat LED
LED_FAULT	Net label	(board-local)	GPIO → fault LED

Note on Power Port Symbols Same convention as the comms board: - **T-bar** for supply rails (+5V_SYS, +3V3, VBAT, VCC_BUS) - **Circle with line** for GND and AGND - The net name on the power port is what matters electrically — symbol shape is cosmetic only.

Note on +3V3 The +3V3 rail on this board is the **Pico module's on-module 3V3 output**, NOT the EPS-supplied VCC from the bus connector. This is different from the comms board (which taps the EPS +3V3 directly). Reason: the FC is small enough that a single Pico module supplies all 3V3 loads on the board (MRAM, WDT, optional CAN transceiver), and we want one well- defined source. VCC_BUS (Pumpkin H2.27/28) is brought to the connector but left unconnected on the FC for v0.1, available for future use if a 3V3-hungry peripheral is added later.

Sheet 1: Overview

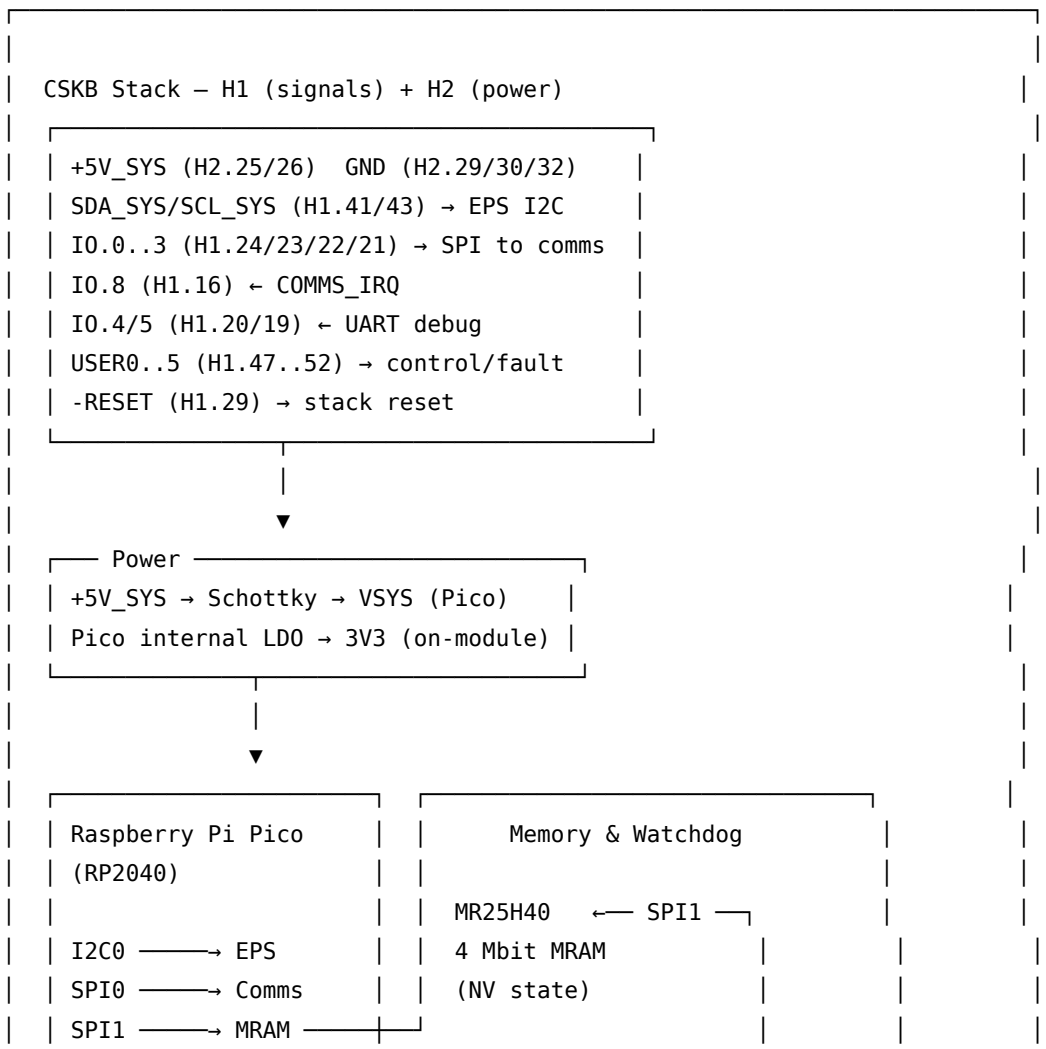
This is a non-electrical title/overview sheet. No components or nets are placed here — it serves as the front page of the schematic package and provides context for anyone reviewing or building the design.

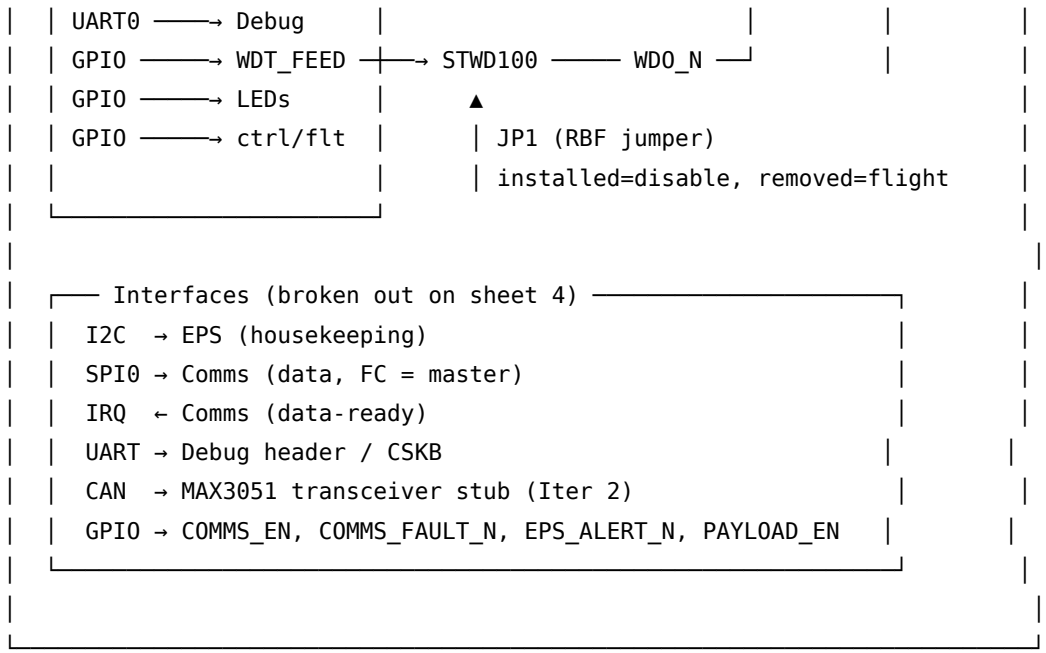
Text Frame 1: Project Identification (Top of Sheet)

Place a large text frame across the top of the sheet:

> **CubeSat Flight Controller Board — RP2040 System Orchestrator**
>
> Project: CentiSat CubeSat — Senior Capstone Design Board: Flight Controller Subsystem, Single PCB Designer: Nick Grabbs / K15Y NG Revision: 0.1 Date: (today) Repository: centisat/hardware/flight_controller

Text Frame 2: System Block Diagram (Center of Sheet)





Text Frame 3: Power Rail Summary (Bottom-Left)

Power Rails

Rail	Source	Voltage	Consumers
+5V_SYS	EPS via CSKB (H2.25, H2.26)	5.0V ±2%	Pico VSYS (via Schottky)
+3V3	Pico on-module LDO output	3.3V ±2%	MR25H40, STWD100, MAX3051 (Iter 2), pull-ups, LEDs
GND	EPS via CSKB (H2.29, H2.30, H2.32)	0V	All
VBAT	EPS via CSKB (H2.45, H2.46)	6.0–8.4V	Not used on FC (EPS owns battery telemetry)
VCC_BUS	EPS via CSKB (H2.27, H2.28)	3.3V (bus)	Reserved, unconnected on FC v0.1

Estimated current budget (3V3, all on-module): RP2040 active: ~30 mA MR25H40 active: ~10 mA, standby <0.1 mA STWD100: <0.5 µA MAX3051 (when populated): ~70 mA peak during transmit LEDs: ~3 mA each (mark DNP for flight) Total: ~50 mA in v0.1 (no CAN), ~120 mA in Iter 2 (CAN active)

+5V_SYS draw: Pico VSYS through Schottky: ~30 mA typical, drops to a few mA in dormant Total stack draw: well within EPS TPS62933F rail capacity

Text Frame 4: Net Name Convention (Bottom-Center)

Net Name Convention

Power rails use **power port symbols** (global across all sheets). Signal nets use **net labels** (also global in flat design).

Net	Type	Description
+5V_SYS, +3V3, GND, VBAT	Power port	Supply rails (global)
I2C_SDA, I2C_SCL	Net label	I2C to EPS LTC4162
SPI_COMMS_*	Net label	SPI bus to comms board (4 nets)
COMMS_IRQ	Net label	Comms→FC data-ready
MRAM_*	Net label	SPI bus to MR25H40 (4 nets, board-local)
UART_DBG_TX, UART_DBG_RX	Net label	Debug UART
COMMS_EN, COMMS_FAULT_N, EPS_ALERT_N, PAYLOAD_EN	Net label	Discrete control/fault GPIOs
CAN_H, CAN_L	Net label	CAN bus (Iter 2)
WDT_FEED, WDT_RST_N	Net label	Hardware watchdog (board-local)
LED_HB, LED_FAULT	Net label	Status LED GPIOs

Text Frame 5: Key Design Notes (Bottom-Right)

Key Design Decisions

1. Power from EPS via CSKB — no on-board regulators (Pico LDO only)
2. Raspberry Pi Pico module for v0.1 (matches comms board pattern)
3. MR25H40 SPI MRAM for persistent state (matches AMSAT RT-IHU)
4. External STWD100 hardware watchdog with RBF-style enable jumper (jumper installed = WDT disabled, removed = flight mode)
5. No on-board RTC — uptime counter in MRAM, ground-synced
6. CSKB pin map adopted verbatim from Pumpkin CubeSat Kit Bus spec for interoperability with commercial Pumpkin/iSpace boards
7. FC is SPI master to comms; COMMS_IRQ signals data-ready
8. CAN stubbed only in v0.1 (MAX3051 transceiver footprint reserved)
9. EPS owns battery telemetry — FC has no on-board VBAT divider
10. Flat schematic design — nets connect by name across sheets

Text Frame 6: Sheet Index (Right Edge)

Sheet Index

Sheet	Title	Contents
1	Overview	This sheet — block diagram, design notes
2	MCU Core	Pico module, decoupling, status LEDs
3	Memory & WDT	MR25H40 MRAM, STWD100 watchdog, RBF jumper
4	Interfaces	I2C, SPI, UART, control/fault GPIOs, CAN stub
5	Power	Input Schottky, bypass, distribution
6	Connectors & Debug	CSKB H1 + H2, debug header, spare GPIO header

Text Frame 7: Revision History

Revision History

Rev	Date	Author	Description
0.1	2026-04-15	NG	Initial schematic — Pico module, MRAM, hardware WDT, Pumpkin CSKB pin map adopted

How to Build This Sheet in Altium

1. Create a new schematic sheet: `Overview.SchDoc`
2. Use Place > Text Frame for each section above
3. For the block diagram, you can either:
   - Use text frames with monospace font (as shown above)
   - Use Place > Drawing Tools > Line/Rectangle to draw boxes and arrows
   - Import a PNG/SVG of the block diagram (Place > Drawing Tools > Graphic)
4. No electrical components, no nets, no power ports on this sheet
5. Set the title block: "CubeSat FC Board — Overview", Rev 0.1

Sheet 2: MCU Core

This sheet contains the Raspberry Pi Pico module, its on-module supply connections, and the board-level status LEDs. All MCU peripheral routing (I2C, SPI, UART, GPIO control/fault) is handled on the Interfaces sheet — this sheet only places the Pico and bypass.

Text Frame (Place > Text Frame, top of sheet):

> MCU CORE — Raspberry Pi Pico module (RP2040-based). Handles command parsing, mode state machine, telemetry aggregation, and watchdog servicing. Using a Pico module for v0.1 — all RP2040 support circuitry (flash, regulator, crystal, USB) is on-module. Bare RP2040 redesign deferred to v2 flight board after architecture is validated. Pico USB is the primary programming and debug interface for v0.1 — no separate USB connector on this board.

Section A: Pico Module (Center of Sheet)

Place a 2×20 pin header symbol (2.54mm pitch) or a Pico module symbol if available in Manufacturer Part Search. Label each pin with its net name.

Pico Module Pin Connections

Pico Pin	Net Name	Goes To	Notes
1 (GP0)	`UART_DBG_TX`	Interfaces sheet → CSKB H1.20	UART0 TX, default debug console
2 (GP1)	`UART_DBG_RX`	Interfaces sheet → CSKB H1.19	UART0 RX
3	`GND`	GND power port	
4 (GP2)	`PAYLOAD_EN`	Interfaces sheet → CSKB H1.50	Payload enable output
5 (GP3)	(spare)	Spare GPIO header on Connectors sheet	Reserved for PAYLOAD_FAULT_N
6 (GP4)	`I2C_SDA`	Interfaces sheet → CSKB H1.41	I2C0 SDA, to EPS
7 (GP5)	`I2C_SCL`	Interfaces sheet → CSKB H1.43	I2C0 SCL, to EPS
8	`GND`	GND power port	
9 (GP6)	`COMMS_IRQ`	Interfaces sheet → CSKB H1.16	Input, internal pull-up
10 (GP7)	`COMMS_EN`	Interfaces sheet → CSKB H1.47	Output, default low
11 (GP8)	`COMMS_FAULT_N`	Interfaces sheet → CSKB H1.48	Input, internal pull-up
12 (GP9)	`EPS_ALERT_N`	Interfaces sheet → CSKB H1.49	Input, internal pull-up
13	`GND`	GND power port	
14 (GP10)	`MRAM_SCK`	Memory_WDT sheet	SPI1 SCK to MRAM

Pico Pin	Net Name	Goes To	Notes
15 (GP11)	`MRAM_MOSI`	Memory_WDT sheet	SPI1 MOSI
16 (GP12)	`MRAM_MISO`	Memory_WDT sheet	SPI1 MISO
17 (GP13)	`MRAM_CS_N`	Memory_WDT sheet	SPI1 CS to MRAM (active-low)
18	`GND`	GND power port	
19 (GP14)	`CAN_CS_N`	Interfaces sheet (Iter 2 stub)	MCP2515 CS, DNP in v0.1
20 (GP15)	`CAN_INT`	Interfaces sheet (Iter 2 stub)	MCP2515 IRQ, DNP in v0.1
21 (GP16)	`SPI_COMMS_MISO`	Interfaces sheet → CSKB H1.22	SPI0 MISO from comms
22 (GP17)	`SPI_COMMS_CS_N`	Interfaces sheet → CSKB H1.24	SPI0 CS to comms
23	`GND`	GND power port	
24 (GP18)	`SPI_COMMS_SCK`	Interfaces sheet → CSKB H1.21	SPI0 SCK
25 (GP19)	`SPI_COMMS_MOSI`	Interfaces sheet → CSKB H1.23	SPI0 MOSI
26 (GP20)	`LED_HB`	Section C below	Heartbeat LED GPIO
27 (GP21)	`LED_FAULT`	Section C below	Fault LED GPIO
28	`GND`	GND power port	
29 (GP22)	`WDT_FEED`	Memory_WDT sheet	STWD100 WDI
30 (RUN)	`WDT_RST_N`	Memory_WDT sheet (open-drain WDO_N from STWD100)	Pico hardware reset, with 10 kΩ pull-up
31 (GP26)	(spare ADC)	Spare GPIO header	ADC0
32 (GP27)	(spare ADC)	Spare GPIO header	ADC1
33	`GND`	GND power port	
34 (GP28)	(spare ADC)	Spare GPIO header	ADC2
35 (ADC_VREF)	—	local 100nF to GND	Internal ADC reference
36 (3V3 OUT)	`+3V3`	+3V3 power port	**Source** of board-level +3V3 — feeds MRAM, WDT, etc.
37 (3V3_EN)	`+3V3`	tied to 3V3 OUT (always enabled)	Internal LDO enable, pulled high to keep 3V3 on
38	`GND`	GND power port	
39 (VSYS)	`VSYS`	Power sheet (from +5V_SYS via Schottky)	Pico system input

Pico Pin	Net Name	Goes To	Notes
40 (VBUS)	(no connect on board)	—	USB VBUS, supplied by Pico's on-module Micro-USB only

Note (Place > Note, next to Pico module):

Pico module power: VSYS is fed from +5V_SYS through a Schottky diode (see Power sheet) so the on-module buck-boost regulator handles all 3V3 generation. The Pico's 3V3(OUT) pin (pin 36) is the **source** of this board's `+3V3` rail — connect it to the +3V3 power port and let it feed MRAM (U2), STWD100 (U3), MAX3051 (U4, DNP), pull-ups, and LEDs. Pin 37 (3V3_EN) is tied to 3V3 OUT to keep the internal regulator always enabled. **Do NOT** connect 3V3 OUT to the bus connector's VCC pin (H2.27/28) — those remain unconnected on FC v0.1. Do NOT connect VBUS (pin 40) to anything on the board; the Pico's own Micro-USB supplies VBUS when plugged in for programming.

Section B: Pico Decoupling

The Pico module has its own on-module decoupling, but additional board-level bulk caps at the 3V3 OUT and GND pins reduce ground bounce when GPIO loads switch.

Ref	Value	Package	Connection	Purpose
C1	10µF, 10V, X5R	0805	+3V3 (Pico pin 36) to GND	Local bulk on Pico 3V3 output
C2	100nF, 16V, X7R	0402	+3V3 to GND, near Pico pin 36	HF bypass
C3	100nF, 16V, X7R	0402	ADC_VREF (pin 35) to GND	ADC reference filter
C4	1µF, 10V, X5R	0603	VSYS (Pico pin 39) to GND	VSYS input bulk

Note (Place > Note, next to decoupling):

C1/C2 catch transients on the on-module 3V3 rail when GPIO loads switch. Place both within 5 mm of pin 36. C3 filters the internal ADC reference — required if any ADC channel is used (we will use them for spare analog headers). C4 sits at VSYS to dampen +5V_SYS connector inductance.

Section C: Status LEDs (Bottom-Right of Sheet)

Ref	Color	Connection	Indicates
D1	Green	+3V3 → R1 (1kΩ) → LED → GND	Board power present
D2	Yellow	GP20 → R2 (330Ω) → LED → GND	Heartbeat (1 Hz blink in nominal mode)
D3	Red	GP21 → R3 (330Ω) → LED → GND	Fault (any active fault flag)

```
+3V3 ── R1 (1kΩ) ── D1 (Green) ── GND        ; always on
GP20 ── R2 (330Ω) ── D2 (Yellow) ── GND      ; heartbeat
GP21 ── R3 (330Ω) ── D3 (Red) ── GND         ; fault
```

Note (Place > Note, next to LEDs):

D1 (power LED) is always-on, pulled directly from +3V3. Mark R1 as DNP-optional in BOM — for flight, leaving the LED unpopulated saves ~1.3 mA continuous draw. D2 (heartbeat) and D3 (fault) are driven by GPIOs and only consume current when on, so they can stay populated for flight. The mode FSM should toggle D2 at 1 Hz in Nominal mode and at 0.25 Hz in Safe mode so a glance at the LED tells you the FC's state.

Sheet 2 Schematic Layout Suggestion

```
┌──────────────────────────────────────────────────────────────────┐
│  [Text Frame: MCU Core Description]                              │
│                                                                  │
│  [Pico Module / 2x20 Header]            [Decoupling]             │
│   GP0  ── UART_DBG_TX (port)            C1 (10µ) C2 (100n)       │
│   GP1  ── UART_DBG_RX (port)            on +3V3 / GND            │
│   GP2  ── PAYLOAD_EN (port)             C3 (100n) on ADC_VREF    │
│   GP4  ── I2C_SDA (port)                C4 (1µ) on VSYS          │
│   GP5  ── I2C_SCL (port)                                         │
│   GP6  ── COMMS_IRQ (port)                                       │
│   GP7  ── COMMS_EN (port)                                        │
│   GP8  ── COMMS_FAULT_N (port)                                   │
│   GP9  ── EPS_ALERT_N (port)                                     │
│   GP10-13 ── MRAM_SCK/MOSI/MISO/CS_N (ports)                     │
│   GP14-15 ── CAN_CS_N / CAN_INT (Iter 2)                         │
│   GP16-19 ── SPI_COMMS_MISO/CS_N/SCK/MOSI (ports)                │
│   GP20 ── LED_HB                                                 │
│   GP21 ── LED_FAULT                                              │
│   GP22 ── WDT_FEED (port)                                        │
```

```
|   RUN  ── WDT_RST_N (port)                                          |
|   GP26-28 ── spare ADC headers                                      |
|   3V3(OUT) ── +3V3 (power port)                                     |
|   3V3_EN ── tied to 3V3(OUT)                                        |
|   VSYS ── from +5V via Schottky (Power sheet)                       |
|   VBUS ── NC                                                        |
|   GND ── GND (power port)                                           |
|                                                                     |
|  [Status LEDs]                                                      |
|   D1(PWR)  D2(HB)  D3(FAULT)                                        |
|                                                                     |
└─────────────────────────────────────────────────────────────────────┘
```

Sheet 2 Ports Checklist

- ☐ `+3V3` — power port (sourced by Pico pin 36)
- ☐ `+5V_SYS` — power port (received via VSYS net from Power sheet)
- ☐ `GND` — power port (global)
- ☐ `VSYS` — net label (to Power sheet Schottky)
- ☐ `I2C_SDA`, `I2C_SCL` — to Interfaces sheet
- ☐ `SPI_COMMS_*` (4 nets) — to Interfaces sheet
- ☐ `COMMS_IRQ`, `COMMS_EN`, `COMMS_FAULT_N`, `EPS_ALERT_N`, `PAYLOAD_EN` — to Interfaces sheet
- ☐ `UART_DBG_TX`, `UART_DBG_RX` — to Interfaces sheet
- ☐ `MRAM_*` (4 nets) — to Memory_WDT sheet
- ☐ `WDT_FEED`, `WDT_RST_N` — to Memory_WDT sheet

Sheet 3: Memory & Watchdog

This sheet contains the persistent storage (MR25H40 4 Mbit SPI MRAM) and the external hardware watchdog (STWD100NYWY3F) with its remove-before- flight enable jumper. Both ICs are on the Pico-sourced +3V3 rail.

Text Frame (Place > Text Frame, top of sheet):

> MEMORY & WATCHDOG — MR25H40 4 Mbit SPI MRAM (U2) provides persistent storage for boot counters, mode state, and fault logs. Unlimited write endurance, no wear levelling needed. STWD100NYWY3F (U3) is an external hardware watchdog that resets the Pico via the RUN pin if the MCU fails to toggle WDT_FEED within the watchdog window. JP1 (Disable Watchdog) is a removable jumper — installed for bench/debug (WDT disabled), removed for flight (WDT enabled). This matches the AMSAT RT-IHU primary_power.kicad_sch RBF watchdog pattern.

Section A: MR25H40 SPI MRAM (Left of Sheet)

Component Search Manufacturer Part Search for "MR25H40" or "MR25H40MDF" (Everspin Technologies, SOIC-8, 3.3V, 40 MHz SPI). Place on the left side of the sheet.

Pin-by-Pin Connections

Pin	Name	Connect To	Notes
1	CS_N	`MRAM_CS_N`	From Pico GP13, active-low
2	SO (MISO)	`MRAM_MISO`	To Pico GP12
3	WP_N	`+3V3` (via 10 kΩ)	Write protect disabled in normal operation; pull-up allows future protect via solder jumper
4	GND	`GND`	
5	SI (MOSI)	`MRAM_MOSI`	From Pico GP11
6	SCK	`MRAM_SCK`	From Pico GP10
7	HOLD_N	`+3V3` (via 10 kΩ)	Hold disabled; tied high through pull-up
8	VDD	`+3V3`	Bypass with 100 nF + 1 µF

Bypass and Pull-ups

Ref	Value	Package	Connection	Purpose
C5	100nF, 16V, X7R	0402	VDD (pin 8) to GND	HF bypass
C6	1µF, 10V, X5R	0603	VDD (pin 8) to GND	Bulk bypass
R4	10 kΩ, 1%	0402	WP_N (pin 3) to +3V3	WP disable pull-up
R5	10 kΩ, 1%	0402	HOLD_N (pin 7) to +3V3	HOLD disable pull-up
R6	10 kΩ, 1%	0402	CS_N (pin 1) to +3V3	Idle-state pull-up so MRAM is deselected at power-on

Note (Place > Note, next to MR25H40):

MR25H40 is a 4 Mbit (512K × 8) magnetoresistive RAM with SPI interface, 3.3V supply, and unlimited write endurance. This matches the U10 (primary) and U24 (secondary) MRAM on AMSAT RT-IHU v1.3 — see `docs/research/rtihu_v1.3.pdf` page 4 (primary_comm_mem sheet). WP_N and HOLD_N are tied high via 10 kΩ pull-ups so the part behaves as a simple SPI memory; if write protection is later required, replace R4 with a zero-ohm to ground via solder jumper. The CS_N pull-up (R6) is critical — it ensures the MRAM is deselected during the brief window between power-on and the Pico driving GP13. Without it, writes to a different SPI device on the same bus could spuriously trigger MRAM operations.

SPI mode: CPOL=0, CPHA=0 (Mode 0). Max clock 40 MHz. We will run this at 8 MHz initially to leave margin for trace inductance.

Section B: STWD100NYWY3F Hardware Watchdog (Center of Sheet)

Component Search Manufacturer Part Search for "STWD100NYWY3F" (ST, SOT-23-5, 3.3V hardware watchdog, ~1.6 s timeout for the NY suffix variant). Place in the center of the sheet.

Pin-by-Pin Connections

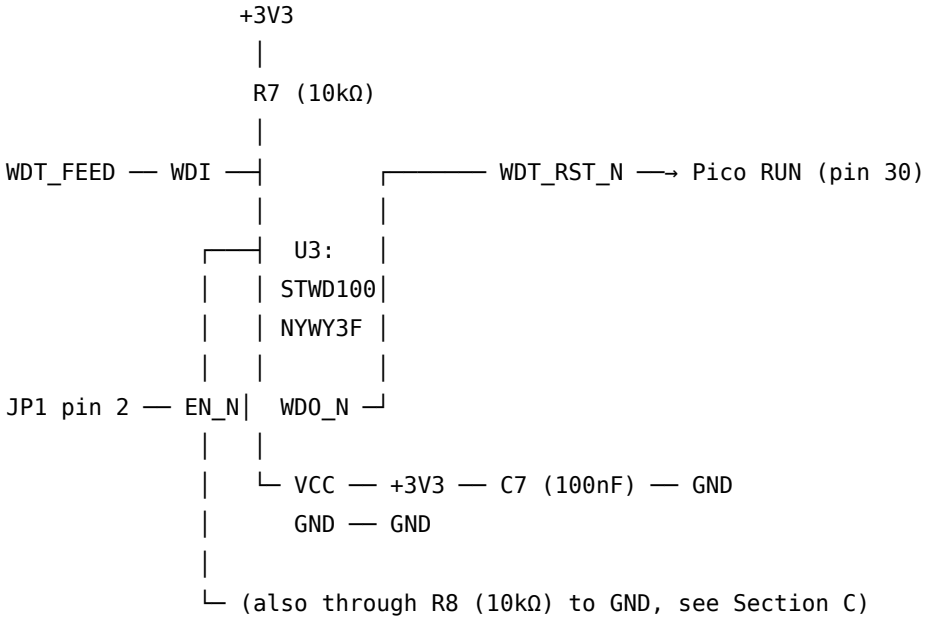
Pin	Name	Connect To	Notes
1	WDI	`WDT_FEED`	From Pico GP22, must toggle at least once per ~1.6 s
2	GND	`GND`	
3	EN_N	JP1 pin 2 (see Section C)	Active-low enable: tied low via JP1 = enabled
4	WDO_N	`WDT_RST_N`	Open-drain output, pulses low on timeout, tied to Pico RUN
5	VCC	`+3V3`	Bypass with 100 nF

Bypass

Ref	Value	Package	Connection	Purpose
C7	100nF, 16V, X7R	0402	VCC (pin 5) to GND	HF bypass

Pico RUN Pull-up

Ref	Value	Package	Connection	Purpose
R7	10 kΩ, 1%	0402	WDT_RST_N to +3V3	Pull-up for Pico RUN line



Note (Place > Note, next to STWD100):

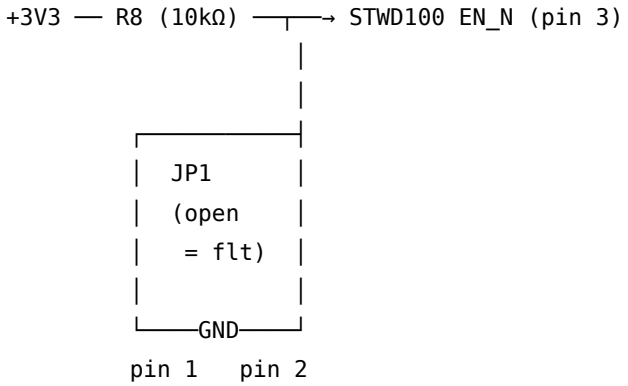
> STWD100NYWY3F watchdog window is ~1.6 seconds (for the NY suffix); if WDT_FEED is not toggled within that window, WDO_N pulses low for ~140 ms which holds Pico RUN low and triggers a hard reset. R7 is the pull-up for the open-drain output AND the Pico's RUN pin — Pico expects RUN to be high for normal operation. The MCU firmware must toggle WDT_FEED (any edge) at least every ~800 ms to be safe — half the window is the standard rule. The reset path is: WDT timeout → WDO_N → RUN low → RP2040 reboot → boot enters Safe mode → MCU re-enables watchdog feed → exits Safe mode after EPS I2C and comms SPI heartbeats valid (per hardware/flight_controller/design/overview.md).

Section C: Watchdog Enable Jumper / RBF (Bottom-Center)

This is the critical bench-vs-flight switch. JP1 is a 1×2 pin header — when a shorting jumper is installed across the two pins, it pulls WDT_EN_BAR low and **disables** the watchdog (bench/debug mode). When the jumper is **removed** ("remove before flight"), the EN_N input is pulled high through R8 and the watchdog is **enabled** (flight mode).

Components

Ref	Part	Package	Connection	Purpose
JP1	1×2 pin header, 2.54mm	THT	Pin 1 = GND, Pin 2 = STWD100 EN_N	Disable Watchdog jumper
R8	10 kΩ, 1%	0402	STWD100 EN_N to +3V3	Pull-up so removed jumper = enabled



State table:

JP1 jumper	EN_N voltage	STWD100 state	Use case
Installed (shorted)	~0 V (pulled low)	DISABLED	Bench / debug — Pico runs free, no spontaneous resets while breakpointing
Removed (open)	+3V3 (via R8)	ENABLED	**Flight mode** — watchdog active, MCU must feed within ~1.6 s

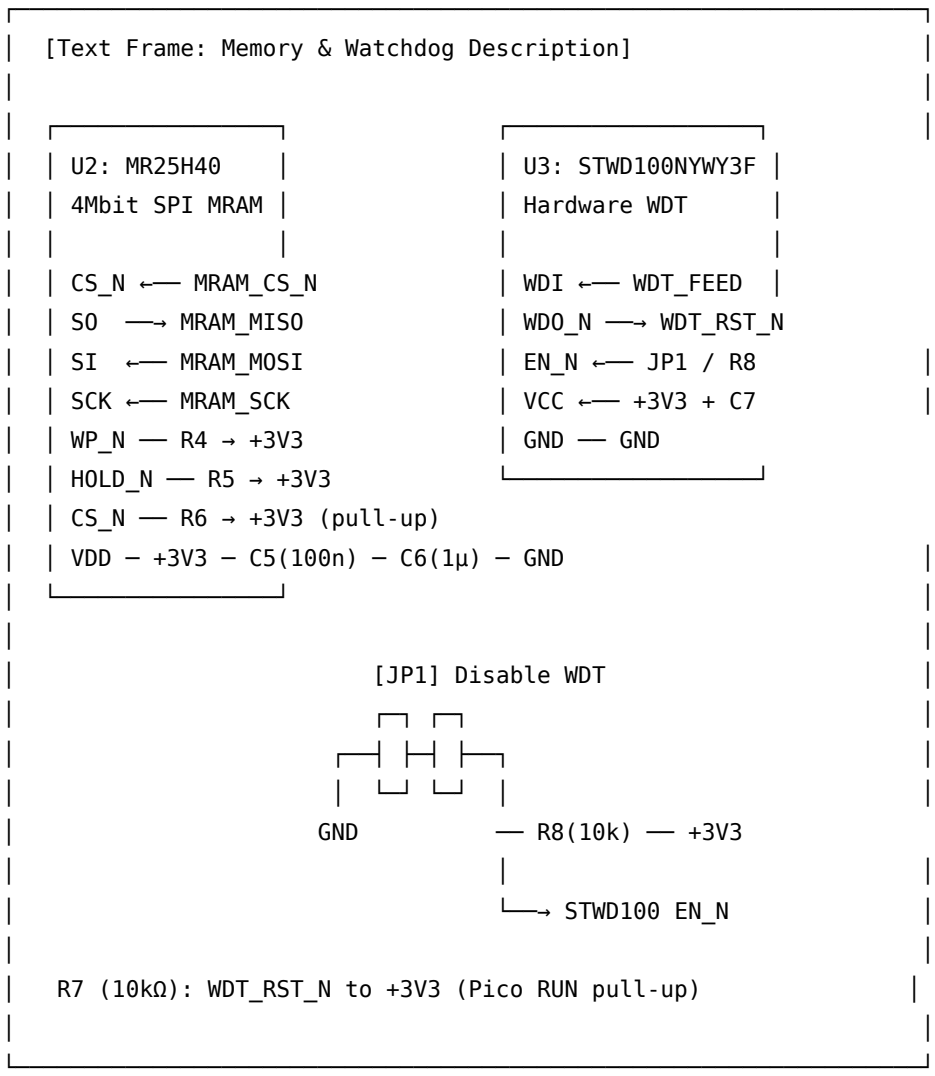
Note (Place > Note, next to JP1, label clearly on silk):

DISABLE WATCHDOG (JP1) — REMOVE BEFORE FLIGHT.

This jumper matches the AMSAT RT-IHU `JP1 Disable Watchdog` implementation on the primary_power.kicad_sch sheet (see `docs/research/rtihu_v1.3.pdf` page 8). Installed = WDT disabled for bench debugging so single-stepping or breakpointing in the debugger doesn't trigger unwanted resets. Removed = flight mode, WDT actively monitors the MCU. The PCB silkscreen MUST clearly label this jumper "DISABLE WDT — REMOVE FOR FLIGHT" so an integration tech doesn't ship the satellite with the WDT bypassed.

R8 (pull-up) ensures the default state with no jumper installed is "enabled" — fail-safe direction. STWD100 EN_N is active-low, so tying it high disables the disable, i.e. enables the watchdog.

Sheet 3 Schematic Layout Suggestion



Sheet 3 Ports Checklist

- ☐ `MRAM_SCK`, `MRAM_MOSI`, `MRAM_MISO`, `MRAM_CS_N` — to MCU Core sheet
- ☐ `WDT_FEED` — to MCU Core sheet (Pico GP22)
- ☐ `WDT_RST_N` — to MCU Core sheet (Pico RUN)
- ☐ `+3V3` — power port (from MCU Core sheet)

☐ `GND` — power port (global)

Sheet 4: Interfaces

This sheet collects all of the off-board signal nets (I2C, SPI, UART, discrete control/fault) and adds the level/pull-up support needed for each, plus the MAX3051 CAN transceiver stub for Iteration 2. The Pico's GPIOs are all driven from the MCU Core sheet — this sheet provides the external support circuitry for those nets and is where they meet the Connectors sheet.

Text Frame (Place > Text Frame, top of sheet):

> INTERFACES — Off-board signal conditioning. I2C bus to EPS LTC4162 (housekeeping telemetry), SPI bus to comms RP2040 (FC = master), UART debug to CSKB backplane / debug header, discrete control and fault GPIOs, and MAX3051 CAN transceiver stub for Iteration 2. All nets here are 3.3V CMOS — no level shifting required because every peer (EPS LTC4162 DVCC, comms RP2040, MAX3051) operates on the same 3.3V domain.

Section A: I2C Bus to EPS (Top-Left)

The EPS board already has its own I2C pull-ups for the LTC4162 (per `hardware/eps/design/altium_eps_schemati`
The FC's role on this bus is master, so we do NOT add a second set of pull-ups here unless the EPS pull-ups prove inadequate during bring-up. Reserve footprints for optional pull-ups.

Optional (DNP) I2C Pull-up Footprints

Ref	Value	Package	Connection	Purpose
R9	4.7 kΩ, 1% (DNP)	0402	`I2C_SCL` to +3V3	Optional pull-up
R10	4.7 kΩ, 1% (DNP)	0402	`I2C_SDA` to +3V3	Optional pull-up

Note (Place > Note):

> R9/R10 are **DNP** by default. The EPS board is the canonical location for I2C pull-ups (see EPS Sheet 3). Populate R9/R10 only if bring-up shows weak edges or marginal timing — but if you do, coordinate with the EPS to avoid excessive parallel pull-up. The shared bus runs at 400 kHz fast mode (per `hardware/flight_controller/design/interfaces.md`).

Section B: SPI to Comms (Top-Right)

Four-wire SPI plus a dedicated interrupt line. FC is master.

Pull-up on COMMS_IRQ The comms RP2040 drives `COMMS_IRQ` as an open-drain or push-pull output. For interoperability, the FC adds a pull-up so the line is defined whenever the comms board is unpowered.

Ref	Value	Package	Connection	Purpose
R11	10 kΩ, 1%	0402	`COMMS_IRQ` to +3V3	Idle-high pull-up

```
SPI_COMMS_SCK   ──→ comms board (CSKB H1.21, IO.3)
SPI_COMMS_MOSI  ──→ comms board (CSKB H1.23, IO.1)
SPI_COMMS_MISO  ←── comms board (CSKB H1.22, IO.2)
SPI_COMMS_CS_N  ──→ comms board (CSKB H1.24, IO.0)
COMMS_IRQ       ←── comms board (CSKB H1.16, IO.8)   + R11 pull-up to +3V3
```

Note (Place > Note):

SPI bus to comms board. FC is master, comms RP2040 is slave. Mode 0 (CPOL=0, CPHA=0), 4–8 MHz initial range per `hardware/flight_controller/design/interfaces.md`. `COMMS_IRQ` mirrors the AMSAT RT-IHU pattern where the AX5043 radio asserts its IRQ line back to the housekeeping unit when a packet is ready (see `/workspace/AMSAT/rt-ihu/drivers/inc/spiDriver.h`). R11 keeps the line defined when the comms board is depowered or in reset. **Comms board v0.2 schematic update required:** add a Pico GPIO output called `COMMS_IRQ` and route it to CSKB pin H1.16 to match this pin map.

Section C: UART Debug (Center-Left)

UART0 from the Pico goes both to a debug header on the Connectors sheet AND to two CSKB H1 pins, so the same UART can be tapped either at a bench probe or via a stack-side USB-to-UART adapter.

```
UART_DBG_TX (from Pico GP0) ──┬──→ Debug header (Connectors sheet)
                              └──→ CSKB H1.20 (Pumpkin IO.4 / UTX0)

UART_DBG_RX (to Pico GP1)  ──┬──← Debug header
                              └──← CSKB H1.19 (Pumpkin IO.5 / URX0)
```

No additional components on this sheet — the UART nets are simply labeled and brought to the appropriate connector pins.

Note (Place > Note):

Debug UART is intentionally exposed on BOTH the bench debug header AND the CSKB backplane so you can leave the FC in the stack and still sniff its console via an adjacent dev board. Default baud 115200 8N1. The Pico's USB-CDC console (over the on-module Micro-USB) is the primary debug channel; this UART is the integration-time backup.

Section D: Discrete Control / Fault GPIOs (Center)

These are simple net labels on this sheet — the GPIOs are driven from the MCU Core sheet and routed to the Connectors sheet. Pull-up resistors on the input lines (active-low fault inputs) ensure a defined state when the peer board is absent or unpowered.

Pull-ups on Active-Low Fault Inputs

Ref	Value	Package	Connection	Purpose
R12	10 kΩ, 1%	0402	`COMMS_FAULT_N` to +3V3	Idle-high (not faulted)
R13	10 kΩ, 1%	0402	`EPS_ALERT_N` to +3V3	Idle-high (not alerting)

The output enables (`COMMS_EN`, `PAYLOAD_EN`) are active-high outputs driven by the Pico — no pull-down is added so the lines float low at power-on (peer boards see "disabled" until FC firmware drives them high). The Pico's GPIOs are tri-stated at reset, but with the peer board's own input pull-down (or its own enable strap), this is safe.

Note (Place > Note):

> Active-low fault inputs (`COMMS_FAULT_N`, `EPS_ALERT_N`) are pulled high through 10 kΩ so the FC sees "no fault" when the peer board is removed, in reset, or unpowered — fail-safe direction. The active-high enable outputs (`COMMS_EN`, `PAYLOAD_EN`) intentionally float low at power-on so peer boards default to disabled until the FC explicitly enables them. This means EPS LTC4162 alert (`EPS_ALERT_N`) is wired here **independently** from the I2C bus — it lets the EPS interrupt the FC without requiring a polled I2C read.

Section E: CAN Transceiver Stub (Bottom-Right) — Iteration 2

This is the CAN transceiver portion of the future CAN node, populated with footprints only in v0.1. The MCP2515 SPI-CAN controller and its crystal are NOT placed on this board in v0.1 — only the transceiver and slew resistor footprints are reserved so v0.2 can populate without a board respin.

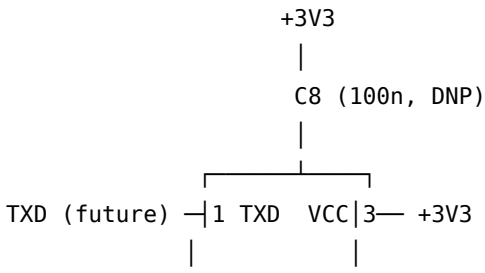
This pattern matches AMSAT RT-IHU's MAX3051 transceiver on `primary_comm_mem.kicad_sch` page 4.

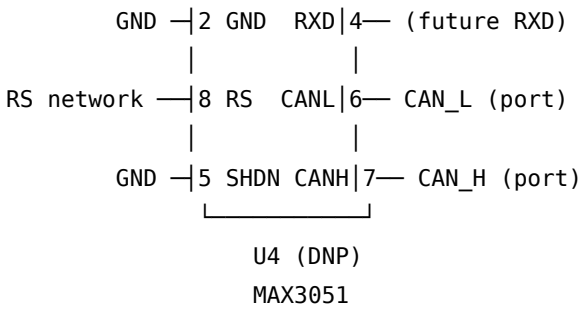
Components (footprints only, DNP for v0.1)

Ref	Value/Part	Package	Connection	Notes
U4	MAX3051	SOIC-8	CAN transceiver (3.3V)	DNP in v0.1
C8	100nF, 16V, X7R	0402	U4 VCC to GND	DNP in v0.1
R14	0Ω	0402	U4 RS pin to R15	DNP, sets slew
R15	24 kΩ, 1%	0402	R14 to GND	DNP, selects 500 kbps slew
R16	120Ω, 1%, 1/4W	0805	`CAN_H` to `CAN_L`	DNP, bus termination (only populate if FC is at the physical end of the CAN bus)

MAX3051 Pin-by-Pin (when populated)

Pin	Name	Connect To	Notes
1	TXD	(future MCP2515 TX)	Tie to pad in v0.1, route to MCP2515 footprint in v0.2
2	GND	`GND`	
3	VCC	`+3V3`	3.3V supply (MAX3051 is the 3V3 variant)
4	RXD	(future MCP2515 RX)	Tie to pad in v0.1
5	SHDN	`GND`	Always active (tied low)
6	CANL	`CAN_L`	To CSKB H1.52 (USER5)
7	CANH	`CAN_H`	To CSKB H1.51 (USER4)
8	RS	`R14/R15` slew network	Selects ~500 kbps slope





R14 (0Ω, DNP) + R15 (24kΩ, DNP) selects 500 kbps slew
R16 (120Ω, DNP) is the optional bus termination

Note (Place > Note, next to MAX3051):

CAN transceiver stub. v0.1 leaves U4, C8, R14, R15, and R16 as unpopulated footprints — board fabricates with the silkscreen and pads but no parts. Iteration 2 will add the MCP2515 SPI-CAN controller (footprint to be added on a v0.2 schematic update, since we want to keep v0.1 small) and populate U4 + R14/R15 to bring the CAN node online. R16 (120Ω termination) only populates if the FC sits at the physical end of the CAN bus topology — typical CubeSat layout puts FC at one end and EPS or comms at the other. The 24 kΩ slew resistor selecting 500 kbps matches AMSAT RT-IHU `R14/R30` on the comm_mem sheet.

MAX3051 is the 3.3V CAN transceiver (vs MAX3050 which is 5V) — be careful when ordering. SOIC-8 package, 8 pins, RS slew control.

Sheet 4 Schematic Layout Suggestion

```
┌──────────────────────────────────────────────────────────────────────┐
│  [Text Frame: Interfaces Description]                                │
│                                                                      │
│  [I2C to EPS]                  [SPI to Comms + IRQ]                  │
│   I2C_SCL ── R9 (DNP) → +3V3   SPI_COMMS_SCK                         │
│   I2C_SDA ── R10 (DNP) → +3V3  SPI_COMMS_MOSI                        │
│                                SPI_COMMS_MISO                        │
│                                SPI_COMMS_CS_N                        │
│                                COMMS_IRQ ── R11 (10k) → +3V3         │
│                                                                      │
│  [UART Debug]                  [Control / Fault GPIOs]               │
│   UART_DBG_TX                  COMMS_EN (out)                        │
│   UART_DBG_RX                  COMMS_FAULT_N ── R12 (10k) → +3V3     │
│                                EPS_ALERT_N   ── R13 (10k) → +3V3     │
```

```
|                                PAYLOAD_EN (out)                          |
|                                                                          |
|  [CAN Transceiver Stub — DNP in v0.1]                                   |
|   U4 MAX3051 (DNP)                                                       |
|   C8 (DNP)                                                               |
|   R14, R15 slew network (DNP)                                            |
|   R16 termination (DNP, only if FC at bus end)                           |
|   CAN_H, CAN_L → CSKB H1.51/52                                           |
|                                                                          |
└──────────────────────────────────────────────────────────────────────────┘
```

Sheet 4 Ports Checklist

- ☐ `I2C_SDA`, `I2C_SCL` — to MCU Core (Pico GP4/GP5) and to Connectors (CSKB H1.41/43)
- ☐ `SPI_COMMS_SCK`, `SPI_COMMS_MOSI`, `SPI_COMMS_MISO`, `SPI_COMMS_CS_N` — to MCU Core (Pico GP18/19/16/17) and to Connectors (CSKB H1.21/23/22/24)
- ☐ `COMMS_IRQ` — to MCU Core (Pico GP6) and to Connectors (CSKB H1.16)
- ☐ `UART_DBG_TX`, `UART_DBG_RX` — to MCU Core (Pico GP0/GP1), to Connectors (CSKB H1.20/19) AND to debug header
- ☐ `COMMS_EN`, `COMMS_FAULT_N`, `EPS_ALERT_N`, `PAYLOAD_EN` — to MCU Core and to Connectors (CSKB USER0..3 = H1.47..50)
- ☐ `CAN_H`, `CAN_L` — to Connectors (CSKB USER4/5 = H1.51/52)
- ☐ `+3V3` — power port
- ☐ `GND` — power port

Sheet 5: Power

This sheet handles power input from the CSKB stack and the Schottky isolation diode that feeds the Pico VSYS. There are NO on-board voltage regulators — the Pico's internal LDO produces all 3V3 on the board.

Text Frame (Place > Text Frame, top of sheet):

> POWER DISTRIBUTION — The flight controller receives +5V_SYS from the EPS via the CSKB stack connectors (H2.25/26). A Schottky diode isolates the stack +5V from the Pico's USB VBUS so the Pico can run from either source without back-feeding. The Pico's on-module buck-boost regulator generates 3V3 from VSYS; that 3V3 (Pico pin 36) is the source of this board's `+3V3` rail — see MCU Core sheet. No on- board voltage regulators on this PCB. Estimated total draw: ~50 mA on +3V3 in v0.1, ~30 mA on +5V_SYS through Schottky.

Section A: Stack +5V_SYS Receive (Left of Sheet)

Power enters via CSKB H2.25/26 (defined on the Connectors sheet). On this sheet we receive the `+5V_SYS` power port and add input bulk plus HF bypass.

Ref	Value	Package	Connection	Purpose
C9	10 µF, 16V, X5R	0805	+5V_SYS to GND	Receiving-end bulk
C10	100 nF, 16V, X7R	0402	+5V_SYS to GND	HF bypass

```
+5V_SYS (from CSKB H2.25/26) ──┬── (to D4 / Schottky)
                              │
                          C9(10µ) C10(100n)
                             │       │
                            GND     GND
```

Note (Place > Note, next to bulk):

C9/C10 are the receiving-end complement to the EPS board's stack- connector bypass caps. Together they form a low-impedance path across the connector. Place both within 5 mm of the +5V_SYS pad on the CSKB H2 connector.

Section B: Schottky Isolation Diode to VSYS (Center)

Ref	Value	Package	Connection	Purpose
D4	Schottky (BAT54 or SS14)	SOT-23 or SMA	`+5V_SYS` (anode) → `VSYS` (cathode)	Isolates stack from Pico USB

```
+5V_SYS ── D4 (Schottky) ──→ VSYS ── (to Pico pin 39)
```

Note (Place > Note, next to D4):

D4 isolates the stack `+5V_SYS` from the Pico's on-module Micro- USB VBUS. When USB is connected (bench programming), the Pico's internal diode on VBUS feeds VSYS. When running from the stack (no USB), D4 feeds VSYS from `+5V_SYS`. Drop across D4 (~0.3 V) gives VSYS ≈ 4.7 V — well within the Pico's VSYS range (1.8–5.5 V). Same pattern as the comms board's D5 (see `hardware/comms/design/altium_comms_schematic.md` Sheet 6, Section C).

SS14 (1A/40V, SMA) is overkill for the FC's ~30 mA draw but it's a JLCPCB basic part and matches the comms board, so reuse it.

Section C: VSYS Bypass at Pico Input

Ref	Value	Package	Connection	Purpose
(uses C4 from MCU Core sheet)	1 µF, 10V, X5R	0603	VSYS to GND, near Pico pin 39	VSYS input bulk

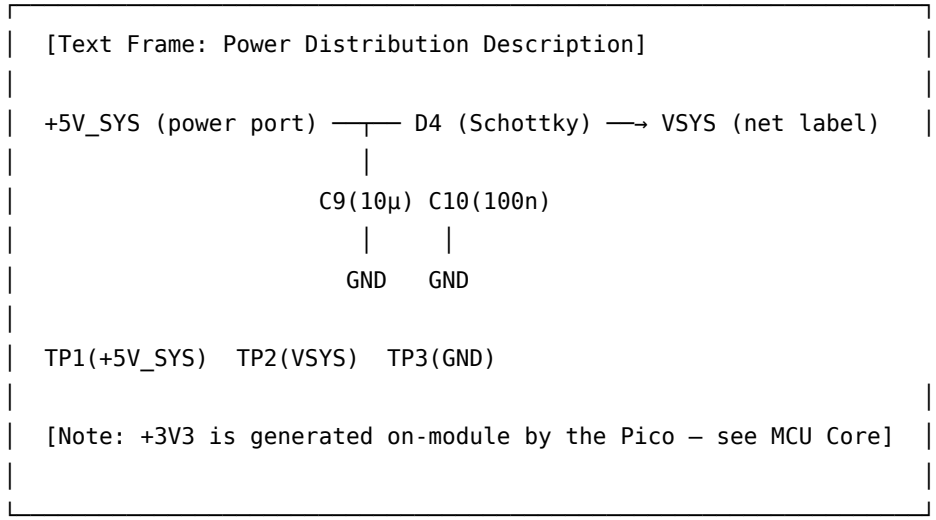
C4 is placed on the MCU Core sheet next to the Pico module so it sits physically next to pin 39 — avoid duplicating it here. The Power sheet just shows `VSYS` continuing into the MCU Core sheet via the net label.

Section D: Power-State Test Points

Ref	Net	Purpose
TP1	`+5V_SYS`	Stack 5 V receive measurement
TP2	`VSYS`	Pico VSYS post-Schottky
TP3	`GND`	Ground reference

(Additional `+3V3` test point is on Sheet 6 next to the Pico module since that's where +3V3 actually originates.)

Sheet 5 Schematic Layout Suggestion



Sheet 5 Ports Checklist

- ☐ `+5V_SYS` — power port (global, from EPS via CSKB H2)
- ☐ `GND` — power port (global)
- ☐ `VSYS` — net label, continues to MCU Core sheet

Sheet 6: Connectors & Debug

This sheet holds the CSKB stack connectors H1 and H2 (using the Pumpkin CubeSat Kit Bus pin map verbatim), the bench debug header, the spare-GPIO header, and the connector-side bypass caps. It is the physical interface between the FC and the rest of the CubeSat stack.

Text Frame (Place > Text Frame, top of sheet):

> CONNECTORS & DEBUG — CSKB stack connectors (H1 for signals, H2 for power) for inter-board power and signal distribution (Pumpkin CSKB pin map), bench debug header for UART + SWD probe access, and spare-GPIO header for unused Pico pins. The FC is a power CONSUMER on the +5V_SYS rail. The H1/H2 pin numbers below are taken verbatim from the Pumpkin CubeSat Kit Motherboard Rev. E datasheet (`DS_CSK_MB_710-00484-E.pdf`, doc Rev. A, March 2012), pages 13–16, so this board is mechanically and electrically compatible with Pumpkin/iSpace CSKB-conformant boards.

Section A: CSKB H1 + H2 Stack Connectors (Center of Sheet)

Connector Selection **Part: 2× Samtec ESQ-126-39-G-D** — 2×26 (52-pin) each, 0.1″ (2.54 mm) pitch, stackthrough through-hole vertical socket. One part populates the H1 (signal) position, one populates the H2 (power) position, for a total of 104 pins per Pumpkin's CSKB layout. The FC uses the **stackthrough** variant (ESQ-126-39-G-D) because it is not the stack endpoint — the EPS is the endpoint and uses the non-stackthrough ESQ-126-37-G-D instead. Default stacking height is 15 mm.

The mating adjacent board's CSKB sockets are the same ESQ-126-39-G-D (or ESQ-126-37-G-D for an endpoint). If taller spacing is needed between specific modules, Samtec SSQ-126-22-G-D 10 mm extension adds up to 24–25 mm between a pair of boards.

Mechanical note: exact X/Y position of the H1 and H2 footprints on the PCB must match Pumpkin's motherboard layout (see `DS_CSK_MB_710-00484-E.pdf` page 5, "Simplified Mechanical Layout") before layout, so a future Pumpkin-compatible board can mate without rework.

Search Manufacturer Part Search for "ESQ-126-39-G-D" or place a generic 2×26 socket symbol (0.1″ (2.54 mm) pitch) and assign the Samtec MPN in the component properties.

Pin Assignment (Pumpkin CSKB-Compatible)

> **Canonical reference:** `system/interfaces/cskb_pinmap.md` is the authoritative pin map for the entire centisat stack. The table below is a board-local view and must match that file. If there is a conflict, the canonical file wins — update this table, not the canonical file.

Pin	Pumpkin name	centisat net	Direction (FC)	Function
H1.16	IO.8	COMMS_IRQ	In	Comms data-ready interrupt (active-low)
H1.19	IO.5 (URX0)	UART_DBG_RX	In	Debug UART RX
H1.20	IO.4 (UTX0)	UART_DBG_TX	Out	Debug UART TX
H1.21	IO.3 (SCK0)	SPI_COMMS_SCK	Out	SPI clock to comms
H1.22	IO.2 (SDI0)	SPI_COMMS_MISO	In	SPI data, comms → FC
H1.23	IO.1 (SDO0)	SPI_COMMS_MOSI	Out	SPI data, FC → comms
H1.24	IO.0 (-CS_SD)	SPI_COMMS_CS_N	Out	SPI chip select to comms
H1.29	-RESET	RESET_N	Bidirectional	Stack reset, tied to Pico RUN via Memory_WDT WDT_RST_N path
H1.41	SDA_SYS	I2C_SDA	Bidirectional	Shared I2C data
H1.42	VBACKUP	(NC v0.1)	—	RTC backup, unused (single pin on CSKB)
H1.43	SCL_SYS	I2C_SCL	Out (master)	Shared I2C clock
H1.47	USER0	COMMS_EN	Out	Comms enable
H1.48	USER1	COMMS_FAULT_N	In	Comms fault
H1.49	USER2	EPS_ALERT_N	In	EPS alert
H1.50	USER3	PAYLOAD_EN	Out	Payload enable
H1.51	USER4	CAN_H	Bus	CAN bus H (Iter 2)
H1.52	USER5	CAN_L	Bus	CAN bus L (Iter 2)
H2.25	+5V_SYS	+5V_SYS	Power In	Stack 5 V rail
H2.26	+5V_SYS	+5V_SYS	Power In	Stack 5 V (parallel for current sharing)
H2.27	VCC_SYS	VCC_BUS	(NC v0.1)	Pumpkin 3V3 bus rail, reserved
H2.28	VCC_SYS	VCC_BUS	(NC v0.1)	(parallel)
H2.29	DGND	GND	Power	Ground return

Pin	Pumpkin name	centisat net	Direction (FC)	Function
H2.30	DGND	GND	Power	(parallel)
H2.31	AGND	AGND	Power	Analog ground, single-point tied to GND (single pin on CSKB)
H2.32	DGND	GND	Power	Ground return (third DGND pin)
H2.45	VBATT	VBAT	(monitor only, NC on FC)	Battery bus
H2.46	VBATT	VBAT	(NC on FC)	(parallel)
(other H1/H2 pins)	—	(NC on FC)	—	Reserved for future use

Note (Place > Note, next to CSKB connectors):

CSKB stack connector pin assignments are taken directly from the Pumpkin CubeSat Kit Motherboard Rev. E datasheet (DS_CSK_MB_710-00484-E.pdf, doc Rev. A, March 2012), pages 13–16. Every pin used on this board carries the same signal name and electrical role as the corresponding Pumpkin CSKB pin, so a future swap-in of a Pumpkin MBM2 or Pumpkin EPS board would Just Work mechanically and electrically. Pins not listed in the table above are NC on the FC v0.1 and remain available for future expansion.

The SD-card SPI lines (IO.0..IO.3 on Pumpkin) are repurposed here as the FC-to-comms SPI bus. Same electrical role (SPI master), different downstream peer — Pumpkin uses these to talk to an on-board SD card socket; we use them to talk to the comms RP2040. Future Pumpkin compatibility is preserved because the CSKB propagates these as plain GPIO from the PPM, and a Pumpkin board would route them however its firmware wants.

EPS_ALERT_N (H1.49) is a centisat-specific addition mapped onto Pumpkin's USER2 pin. EPS LTC4162's SMBALERT_N from the EPS Sheet 1 is wired to this same H1 pin on the EPS board, so the line runs straight through the stack without any FC-side translation.

Bulk Bypass at Stack Connector Place bypass caps physically close to the H2 connector pads. Same convention as the comms board.

Ref	Value	Package	Connection	Purpose
C11	10 µF, 16V, X5R	0805	+5V_SYS to GND at connector	Receiving-end bulk
C12	100 nF, 16V, X7R	0402	+5V_SYS to GND at connector	Receiving-end HF

Note (Place > Note, next to bypass caps):

Connector-side bypass on +5V_SYS only. We do NOT add bypass on VCC_BUS (Pumpkin VCC / 3V3 bus) because the FC does not consume from that rail in v0.1 — the Pico generates its own 3V3. If a future revision adds a VCC_BUS load, add receiving caps then.

Section B: Bench Debug Header (Bottom-Left)

A 1×6 header gives bench access to UART + SWD without needing the stack connector or USB.

Ref	Part	Pins	Connection
J2	1×6 pin header, 2.54 mm	6	GND, +3V3, UART_DBG_TX, UART_DBG_RX, SWCLK, SWDIO

Pin	Net	Notes
1	GND	Reference
2	+3V3	Power for serial adapter logic side
3	UART_DBG_TX	From Pico GP0
4	UART_DBG_RX	To Pico GP1
5	SWCLK	Pico SWD clock
6	SWDIO	Pico SWD data

Note (Place > Note, next to J2):

Standard 6-pin debug header — pin 1 marked with a square pad on silkscreen. Compatible with Adafruit/SparkFun FTDI cables for UART (use pins 1–4) and with Pi Pico debug probes for SWD (use pins 1, 2, 5, 6). Both flavors share the +3V3 and GND pins. The Pico's SWCLK/SWDIO pins are on the underside of the module — route them via the bottom row of the 2×20 pin landing (Pico debug pins 1 and 2 of the small 3-pin debug connector).

Section C: Spare GPIO Header (Bottom-Center)

A 1×8 header brings every Pico GPIO that has no fixed function to a probe-friendly location. Useful for bring-up, integration tape-up, and adding sensors during testing.

Ref	Part	Pins	Connection
J3	1×8 pin header, 2.54 mm	8	GP3, GP14, GP15, GP26, GP27, GP28, GND, +3V3

Pin	Net	Notes
1	GP3	Reserved for PAYLOAD_FAULT_N future use
2	GP14	Reserved for CAN_CS_N (Iter 2)
3	GP15	Reserved for CAN_INT (Iter 2)
4	GP26	Spare ADC0
5	GP27	Spare ADC1
6	GP28	Spare ADC2
7	GND	
8	+3V3	

Note (Place > Note, next to J3):

Spare GPIO header — invaluable during integration. Pins 1–3 are earmarked for future signals (PAYLOAD_FAULT_N, CAN_CS_N, CAN_INT) but are exposed here so they can be temporarily repurposed during bring-up. Pins 4–6 are the three remaining ADC-capable pins on the RP2040 — bring sensors (temperature, current shunt, etc.) here for testing without a board respin. Label each pin on the silkscreen with its GP number so the technician doesn't need to consult the schematic.

Section D: Test Points (Bottom-Right)

Ref	Net	Purpose	Measurement
TP4	+3V3	Pico on-module 3V3 output	Rail voltage, ripple
TP5	VSYS	Pico VSYS (post-Schottky)	Stack 5 V receive
TP6	GND	Ground reference	Loop-style for scope clip
TP7	RESET_N / Pico RUN	Hardware reset line	Verify WDT reset behavior
TP8	SPI_COMMS_SCK	SPI clock	Logic analyzer / scope
TP9	SPI_COMMS_MOSI	SPI MOSI	
TP10	SPI_COMMS_MISO	SPI MISO	

Ref	Net	Purpose	Measurement
TP11	SPI_COMMS_CS_N	SPI CS	
TP12	I2C_SCL	EPS I2C clock	
TP13	I2C_SDA	EPS I2C data	
TP14	COMMS_IRQ	Comms interrupt	
TP15	WDT_FEED	Watchdog feed	Verify ~1 Hz toggle
TP16	GND	Second ground reference (opposite side of board)	

How to Place Test Points in Altium

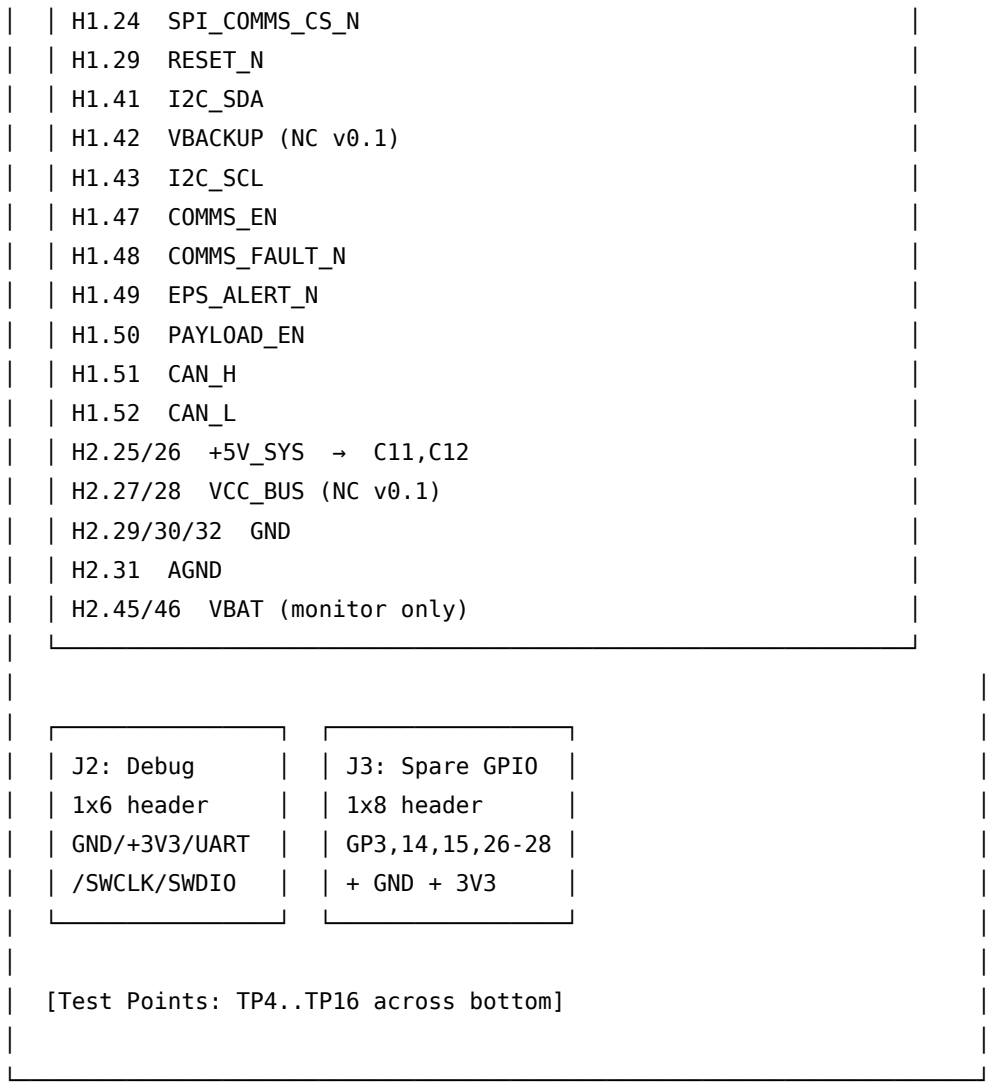
1. Place > Part > search "Test Point" in Manufacturer Part Search
2. Or: create a generic 1-pin schematic symbol called "TP"
3. Assign a footprint: round pad (1–2 mm diameter) or Keystone 5019 loop
4. Label each test point with its net name in the Comment field

Note (Place > Note, next to test points):

TP4–TP6 are the absolute minimum test points required for any bring-up — power, VSYS, and GND. TP7 (RUN/RESET_N) lets you verify the watchdog actually resets the MCU on JP1 removal + intentional feed timeout. TP8–TP14 expose every off-board signal for logic-analyzer captures during integration. TP15 lets you watch the watchdog feed pulse on a scope to confirm firmware is servicing it. TP6 and TP16 are both ground; place them on opposite ends of the board so a probe always has a short ground clip path.

Sheet 6 Schematic Layout Suggestion

```
┌──────────────────────────────────────────────────────────────────────┐
│  [Text Frame: Connectors & Debug Description]                        │
│                                                                      │
│  ┌──────────────────────────────────┐                                │
│  │ J1 (H1) + J1 (H2): 2× ESQ-126-39-G-D                          │
│  │ CSKB Stack Connectors (2×26 each, 0.1″)                        │
│  │                                                                 │
│  │ H1.16  COMMS_IRQ                                                │
│  │ H1.19  UART_DBG_RX                                              │
│  │ H1.20  UART_DBG_TX                                              │
│  │ H1.21  SPI_COMMS_SCK                                            │
│  │ H1.22  SPI_COMMS_MISO                                           │
│  │ H1.23  SPI_COMMS_MOSI                                           │
```



Sheet 6 Ports Checklist

- ☐ All Pumpkin H1/H2 nets per the table above
- ☐ `+5V_SYS` — power port from CSKB H2
- ☐ `+3V3` — power port (sourced by Pico, used here for debug header pin 2)
- ☐ `GND` — power port from CSKB H2

After All Six Sheets Are Done

1. Annotate

Tools > Annotate Schematic Quietly (or Annotate Schematic for manual control). Assign unique designators across all sheets: R1, R2, ... C1, C2, ... U1, U2, ... etc.

Expected designator counts: - Resistors: ~16 (R1–R16, including DNP) - Capacitors: ~12 (C1–C12) - ICs: U1 (Pico module), U2 (MR25H40), U3 (STWD100), U4 (MAX3051, DNP) - Diodes: D1 (power LED), D2 (heartbeat LED), D3 (fault LED), D4 (Schottky) - Connectors: J1 (CSKB stack, implemented as 2× ESQ-126-39-G-D for H1 and H2), J2 (debug header), J3 (spare GPIO header) - Jumpers: JP1 (Disable Watchdog) - Test points: TP1–TP16

2. Compile and Validate

Project > Validate PCB Project. Check the Messages panel for: - **Net mismatch** — net labels with the same name on different sheets must match exactly (case-sensitive). Watch out for `SPI_COMMS_*` vs `SPI_COMM_*` typos and `COMMS_IRQ` vs `COMM_IRQ`. - **Unconnected pins** — add no-connect X markers on intentionally unconnected pins (e.g., Pico VBUS, MAX3051 TXD/RXD in v0.1, all unused CSKB pins). - **Duplicate designators** — fix before proceeding (annotate should handle this). - **Power port direction warnings** — review to ensure power ports are connected, not floating.

3. Fill in Title Blocks

On each sheet, double-click the title block and fill in:

Field	Sheet 1	Sheet 2	Sheet 3
Title	FC — Overview	FC — MCU Core	FC — Memory & WDT
Revision	0.1	0.1	0.1
Drawn By	Nick Grabbs / K15Y NG	Nick Grabbs / K15Y NG	Nick Grabbs / K15Y NG
Date	(today)	(today)	(today)

Field	Sheet 4	Sheet 5	Sheet 6
Title	FC — Interfaces	FC — Power	FC — Connectors & Debug
Revision	0.1	0.1	0.1
Drawn By	Nick Grabbs / K15Y NG	Nick Grabbs / K15Y NG	Nick Grabbs / K15Y NG
Date	(today)	(today)	(today)

4. Export PDF

File > Smart PDF (or File > Print > PDF). Save to `hardware/flight_controller/releases/FC_schematic_v0.1.p` for review.

Complete BOM Summary

Sheet 2: MCU Core

Ref	Part	Package	Notes
U1	Raspberry Pi Pico module	2×20 SMD/THT	RP2040-based, on-module flash + LDO
C1	10 µF, 10V, X5R	0805	Pico 3V3 OUT bulk
C2	100 nF, 16V, X7R	0402	Pico 3V3 OUT HF
C3	100 nF, 16V, X7R	0402	ADC_VREF filter
C4	1 µF, 10V, X5R	0603	VSYS bulk
R1	1 kΩ, 1% (DNP-optional for flight)	0402	Power LED current limit
R2	330 Ω, 1%	0402	Heartbeat LED current limit
R3	330 Ω, 1%	0402	Fault LED current limit
D1	LED green	0603	Power indicator
D2	LED yellow	0603	Heartbeat
D3	LED red	0603	Fault

Sheet 3: Memory & Watchdog

Ref	Part	Package	Notes
U2	Everspin MR25H40 (4 Mbit SPI MRAM)	SOIC-8	Persistent state
U3	ST STWD100NYWY3F (hardware WDT)	SOT-23-5	~1.6 s window
C5	100 nF, 16V, X7R	0402	MRAM HF bypass
C6	1 µF, 10V, X5R	0603	MRAM bulk
C7	100 nF, 16V, X7R	0402	STWD100 bypass
R4	10 kΩ, 1%	0402	MRAM WP_N pull-up
R5	10 kΩ, 1%	0402	MRAM HOLD_N pull-up
R6	10 kΩ, 1%	0402	MRAM CS_N idle pull-up
R7	10 kΩ, 1%	0402	Pico RUN / WDO_N pull-up
R8	10 kΩ, 1%	0402	STWD100 EN_N pull-up (default to enabled)
JP1	1×2 pin header, 2.54 mm	THT	Disable Watchdog (RBF)

Sheet 4: Interfaces

Ref	Part	Package	Notes
U4	Maxim MAX3051 (3.3V CAN transceiver)	SOIC-8	**DNP in v0.1**
C8	100 nF, 16V, X7R	0402	MAX3051 bypass, **DNP in v0.1**
R9	4.7 kΩ, 1%	0402	Optional I2C SCL pull-up, **DNP**
R10	4.7 kΩ, 1%	0402	Optional I2C SDA pull-up, **DNP**
R11	10 kΩ, 1%	0402	COMMS_IRQ idle pull-up
R12	10 kΩ, 1%	0402	COMMS_FAULT_N idle pull-up
R13	10 kΩ, 1%	0402	EPS_ALERT_N idle pull-up
R14	0 Ω	0402	MAX3051 RS slew, **DNP in v0.1**
R15	24 kΩ, 1%	0402	MAX3051 RS slew (500 kbps), **DNP in v0.1**
R16	120 Ω, 1%, 1/4W	0805	CAN bus termination, **DNP** unless FC at bus end

Sheet 5: Power

Ref	Part	Package	Notes
C9	10 µF, 16V, X5R	0805	+5V_SYS receive bulk
C10	100 nF, 16V, X7R	0402	+5V_SYS receive HF
D4	SS14 or BAT54 (Schottky)	SMA or SOT-23	+5V_SYS → VSYS isolation

Sheet 6: Connectors & Debug

Ref	Part	Package	Notes
J1	2× Samtec ESQ-126-39-G-D	2×26, 0.1″ (2.54 mm) stackthrough (×2, one per H1 and H2)	CSKB stack connectors

Ref	Part	Package	Notes
J2	1×6 pin header, 2.54 mm	THT	Debug header (UART + SWD)
J3	1×8 pin header, 2.54 mm	THT	Spare GPIO header
C11	10 µF, 16V, X5R	0805	CSKB H2 +5V_SYS bulk
C12	100 nF, 16V, X7R	0402	CSKB H2 +5V_SYS HF
TP1–TP16	Test point pads	1.5 mm round	Various nets per Section D

v0.1 Populated-Parts Count (excluding DNP)

- 16 capacitors (C1–C7, C9–C12 + 0 DNP) — actually 11 populated, 1 DNP (C8)
- 9 resistors (R1–R3, R6, R7, R8, R11, R12, R13) — 7 DNP (R4, R5 are populated wait... let me recount)

Wait — recount populated resistors: - Populated: R1 (LED, optional DNP for flight), R2, R3, R4, R5, R6, R7, R8, R11, R12, R13 = 11 - DNP: R9, R10 (I2C optional), R14, R15, R16 (CAN stub) = 5

- 4 ICs: U1 (Pico), U2 (MRAM), U3 (WDT) populated; U4 (CAN) DNP
- 4 diodes: D1, D2, D3, D4 populated
- 3 connectors + 1 jumper: J1, J2, J3, JP1
- 16 test points

Total populated component count: ~38 parts (very compact for a flight controller) Total board area target: <50 cm² (CSKB / CubeSat Kit footprint is the constraint)

Outstanding Items / Follow-ups

1. **Comms board v0.2 update required.** Add `COMMS_IRQ` to the comms board's Digital_Control sheet (pick one Pico GPIO, e.g. GP22) and route it to CSKB H1.16 on the Connectors sheet. One-line update to `system/interfaces/comms_to_fc.md` to document the signal.
2. **EPS board v0.2 follow-up.** The EPS LTC4162 SMBALERT_N must route to CSKB H1.49 (Pumpkin USER2) so it lands on the FC's `EPS_ALERT_N` net. Verify the EPS schematic reflects this when next touched.
3. **CSKB connector layout.** v0.1 uses 2× Samtec ESQ-126-39-G-D (stackthrough, 52 pins each) for H1 and H2, matching Pumpkin's CubeSat Kit Bus exactly. EPS is the stack endpoint and uses 2× ESQ-126-37-G-D (non-stackthrough). If any module is later replaced by a Pumpkin MBM2/EPS, no connector change is needed. Stacking height is 15 mm v0.1; Samtec SSQ-126-22-G-D can be inserted for 24–25 mm spacing if a module needs clearance. See `system/interfaces/cskb_pinmap.md` for the canonical part catalog.

4. **CAN controller (Iteration 2).** v0.2 of this schematic will add the MCP2515 SPI-CAN controller, its 16 MHz crystal, and route GP14/GP15 to it. Plan to populate U4 (MAX3051) and the slew network at the same time.
5. **Bring-up plan reference.** See `hardware/flight_controller/bringup/phase1_validation.md` for the phase-1 acceptance procedure. Key checks: power-on with WDT disabled, then power-on with WDT enabled (verify 1 Hz feed), then I2C read of LTC4162, then SPI loopback to comms.
